

Solutions to the challenges proposed by 0xPARC

Exact representations, index deconvolution and constrained computation

Alberto Hernandez-Espinosa

17 September 2026

Abstract

The 0xPARC challenges ask how much can be done with a small set of permitted operations: recover a hidden list, construct a large majority vote, transform a packed vector and express integer conditions through quadratic equations. I present possible solutions to all eight questions and examine what their representations preserve. An index-deconvolution method recovers a Boolean system’s functional description from its behaviour alone; run on a canonical decision diagram it does so without enumerating states, which takes the exact recovery of a majority circuit to 151 inputs. The same method supplies the gates for the arithmetic questions: each cell is stated as an integer relation and whatever gate the method returns is what is compiled, so the classical full adder is recovered rather than assumed. Each answer states which of three roles the method plays for it, including the one question where it supplies only a measured limit. For the Fourier problem, grouping stages into three multiplication levels reduces the stated operation cost by factors of 6.347 and 4.743 relative to a dense construction at the two stated dimensions. For factorisation, explicit bounds make the modular constraints equivalent to integer multiplication. The main text explains the constructions through worked examples; Supplementary Information gives the proofs, complete specifications and verification evidence.

1 Disclaimer on the use of AI

AI tools were used as assistants for constructing the testbed, searching for and corroborating bibliography and related work, discussing the solutions presented here, and reviewing grammar and phrasing. The ideas, research, methods, validation, interpretations and claims are original to the author, who takes responsibility for the final content and for independently checking the cited sources.

2 What index deconvolution does for each answer

Index deconvolution takes the complete finite input–output repertoire of a Boolean system and returns, for each output, the inputs that actually matter and the gate acting on them. Single-bit perturbations isolate the responsive coordinates; the restricted behaviour is then matched against a canonical gate family. The recovered description is replayed forward and must reproduce the original behaviour exactly. This follows the algorithmic-information-dynamics practice of using interventions and generative descriptions to expose structure [2, 3].

A method is only as useful as the claims it actually carries, so each answer below states which of three roles the method plays. It **derives** an answer when the object compiled into that answer is what the method returned. It **certifies** an answer when the answer is argued another way and the method recovers and verifies a decisive Boolean part of it exactly. It **bounds** an answer when it does not produce the answer at all and what it contributes is a measurement of why.

	Role	What the method supplies
Q1	certifies	every decoder output bit reads one position of its own digit window
Q2	derives	majority recovered to 151 inputs, identical to an independent threshold
Q3	certifies	the emitted schedule’s dependency support, recovered and shown total
Q4	certifies	the solution-set structure for each tractable prime
Q5	derives	the comparator the range predicate is built from
Q6	derives	the comparator and inequality cells at 254 bits
Q7	derives	the full adder and partial product the 64-bit multiplier is built from
Q8	bounds	a measurement of where the representation stops working

Two consequences are worth stating plainly at the outset. The arithmetic cells are not written down anywhere in the source: each is stated as an integer relation, handed to the method, and whatever gate comes back is what gets compiled. For the full adder the method returns sum as XOR and carry as MAJORITY without either being named. And Q8’s answer is the direct limb construction of Section 8.3; the method does not produce it, and Section 8.3 reports the measurement that explains why. Supplementary Sections S10–S11 give the evidence, the denominators and the limits.

3 Introduction

The challenges, presented through 3Blue1Brown, bring together several kinds of restricted computation [1]. A hidden list can be queried but not read. A majority vote must be assembled from three-input gates. A Fourier transform has only three multiplication levels available. Integer conditions must be expressed by equations in a finite field. In each case, the first question is whether the permitted operations can express the answer. A second question is what a particular construction costs and how its correctness can be checked.

I answer the questions in their original order, numbered Q1–Q8. The mathematical ingredients include positional encoding, majority identities, Fourier factorisation and binary arithmetic. These have established precedents [4, 5]. The contribution of this response is to connect the solutions to explicit representations that can be checked in both directions, and to be exact about which direction carries which claim.

Running an answer forwards and recovering it backwards are different acts. A circuit tells us how to compute an output. Its repertoire is the complete table of outputs over all input states, and index deconvolution starts from that table to ask which inputs matter and what function they compute. Where the recovered description is what we then compile, the method has done the work. Where the answer rests on a proof over unbounded integers, or on complex arithmetic, or on the field’s own multiplication, it has not, and the table in Section 2 says so.

One practical point decides how far any of this reaches. Deconvolution was originally defined over a materialised repertoire, which costs 2^n and caps every claim built on it at a dozen or so inputs. The same three steps run instead on an ordered decision diagram, where a canonical representation makes structural identity the same thing as agreement on all 2^n inputs. Essential variables become a comparison of two restrictions, gate naming becomes an identity test, and verification becomes an identity test on the recompiled system. Nothing is sampled and nothing is approximated; what disappears is the need to write the repertoire down. That is why Q2

below reaches 151 inputs rather than eleven, and it is also why Q8 does not reach at all.

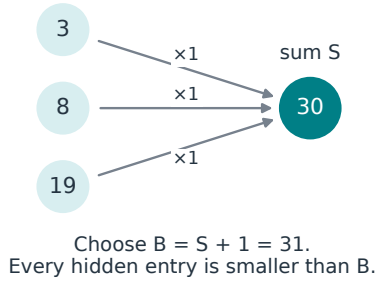
4 Recovering a hidden list (Q1)

Suppose the hidden list contains positive integers $v_0, \dots, v_{n-1}$. A query specifies coefficients, and the answer is their exact dot product with the list. First ask for its sum, $S = \sum_i v_i$. For $n > 1$, choose $B = S + 1$ and ask a second question with coefficients $1, B, \dots, B^{n-1}$:

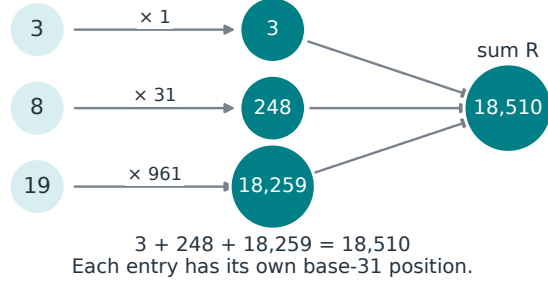
$$R = \sum_{i=0}^{n-1} v_i B^i. \quad (1)$$

Each entry is smaller than B , so the answer stores the list as base- B digits. Divide by B , retain the remainder and repeat. Each remainder is one entry of the list. Figure 1 shows how each entry contributes to the two answers and how the second answer is unpacked.

A. First query: give every entry weight 1



B. Second query: assign distinct digit weights



C. Divide by 31: pass the quotient onward and keep the remainder

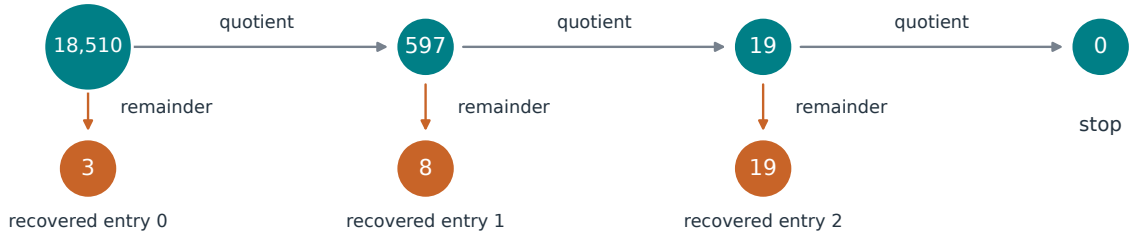


Figure 1: How do two queries recover the hidden entries? Panel A gives each entry weight one, returning $S = 30$ and allowing base $B = 31$. Panel B weights the same entries by $1, 31, 961$ and adds their contributions to obtain $R = 18510$. Panel C divides by 31 at each step: the quotient continues along the network, while the orange remainder is one recovered entry. The hidden values are shown to explain the example; the querying party observes only the two answers. Exact digits are required.

If an upper bound U is known, choose $B = U + 1$ immediately and use one query. A one-entry list also needs only one query. For unbounded lists of length at least two, one fixed rational query cannot suffice: there is a nonzero integer change to the list whose dot product with the coefficients is zero. Starting with sufficiently large positive entries gives two positive lists with the same answer. Thus two adaptive rational queries are necessary and sufficient.

The phrase “any list of numbers” also admits ideal exact real coefficients. Coefficients linearly independent over the rationals distinguish every integer list in one query. For example, powers of a transcendental number have this property. This establishes uniqueness under exact real

arithmetic; it does not give a finite-precision decoding procedure. Fewer questions can require more information in each answer. Supplementary Section S2 gives the full argument and its assumptions.

5 Constructing and recovering majority (Q2)

A three-input majority gate returns one when at least two inputs are one. The question asks whether these gates can compute majority on 2025 inputs, with free duplication of wires. They can. The useful identity is easiest to understand by temporarily allowing a more general monotone Boolean function f : changing an input from zero to one never lowers its output.

Choose three distinct inputs a, b, c . Form three simpler functions by replacing b with a , c with b , and a with c , respectively. Then

$$f = \text{MAJ}_3(f_{b \leftarrow a}, f_{c \leftarrow b}, f_{a \leftarrow c}). \quad (2)$$

If the three input bits agree, all substitutions preserve the assignment. Otherwise, one preserves it, one increases a bit and one decreases a bit. Their outputs therefore bracket the original output, with the original itself among them. Their majority equals f .

Each substitution removes an active input. Odd majority is also self-dual: complementing every input complements the output. Both properties survive substitution, and a monotone self-dual function on at most two inputs is just an input wire. Repeating Eq. (2) therefore produces a circuit containing only three-input majority gates. The proof applies to every odd input count, including 2025. It gives existence; its loose size bound does not establish an economical circuit at that scale.

5.1 A network we can inspect completely

For five inputs, the construction emits the seven gates in Figure 2. Each arrow carries an input bit or the output of an earlier gate. For example, $g_0 = \text{MAJ}_3(x_0, x_2, x_4)$ and $g_1 = \text{MAJ}_3(x_0, g_0, x_3)$. The final gate takes g_1, g_3, g_5 . Reading across a row of the connection matrix identifies the three sources of that gate; evaluating the gates in order gives the output.

Take input 11100, written in the order $x_0, \dots, x_4$. Its first three bits are one, and its output is one. Now vary the last two bits. The four inputs 11100, 11110, 11101, 11111 still have output one, although some internal gates change. This is the pattern 111*: either value is allowed at each star. The network shows how the answer is produced, while the pattern identifies which variations preserve that answer.

5.2 What index deconvolution adds

A circuit describes how to compute an output. Its repertoire is the complete table of outputs for all input states. Index deconvolution starts from that table and asks which inputs actually affect the result. For a Boolean function f on n bits, encode a state by

$$\iota(x) = \sum_{i=0}^{n-1} 2^i x_i, \quad \mathcal{I}_f = \{\iota(x) : f(x) = 1\}. \quad (3)$$

The index set $\mathcal{I}_f$ records exactly where the output is one; all other indices have output zero. Input i matters if there is a pair of states differing only at i for which the outputs differ. These pairs isolate the dependence of this output condition inside the complete repertoire. They do not imply that the integrated network or its evolving behaviour has been reduced to a smaller table:

A. Follow the actual majority circuit



Every gate returns the majority of its three incoming wires.

B. Read one target row

Target gate	x_0	x_1	x_2	x_3	x_4	g_0	g_1	g_2	g_3	g_4	g_5
g_0	1	0	1	0	1	0	0	0	0	0	0
g_1	1	0	0	1	0	1	0	0	0	0	0
g_2	0	1	0	1	1	0	0	0	0	0	0
g_3	1	1	0	0	0	0	0	1	0	0	0
g_4	0	0	1	1	1	0	0	0	0	0	0
g_5	0	1	1	0	0	0	0	0	0	1	0
g_6	0	0	0	0	0	0	1	0	1	0	1
	x_0	x_1	x_2	x_3	x_4	g_0	g_1	g_2	g_3	g_4	g_5

Source input or gate

C. The same input pattern can contain different internal gate states

Index	Input $x_0 \dots x_4$	g_0	g_1	g_2	g_3	g_4	g_5	g_6	Output
7	11100	1	1	0	1	0	1	1	1
15	11110	1	1	1	1	1	1	1	1
23	11101	1	1	1	1	1	1	1	1
31	11111	1	1	1	1	1	1	1	1

Figure 2: How does a network of small majority gates compute a larger vote? Panel A shows the actual five-input, seven-gate construction, with arrows from sources to targets. Every gate computes three-input majority. Panel B records the same connections as a matrix; each target row has three ones. Panel C evaluates all four fillings of 111 **. Some internal outputs change while the final output remains one. Inputs read $x_0 \dots x_4$; their index weights are 1, 2, 4, 8, 16.

changing the selected node union or dynamical condition changes the observation being asked of the same system. Equality with the original repertoire checks that the stated finite recovery is exact.

A further compression describes groups of output-one states by patterns, or *schemata*. Figure 3 follows the same five-input network through its ten schemata and all 32 input states. Each schema fixes three bits to one and leaves two free. Any state with at least three ones belongs to one or more of these patterns; no state with fewer ones belongs to any of them.

For $111**$, the fixed ones give decimal anchor $1 + 2 + 4 = 7$. The *sumandos* are the fillings of the schema's don't-care positions. Here their numerical offsets are 0, 8, 16, 24: set neither free bit, only bit 3, only bit 4, or both. Adding these offsets to 7 gives indices 7, 15, 23, 31, the four highlighted rows of the repertoire. The ten schemata together expand to exactly 16 output-one indices. The union counts each index once, even when several schemata contain it.

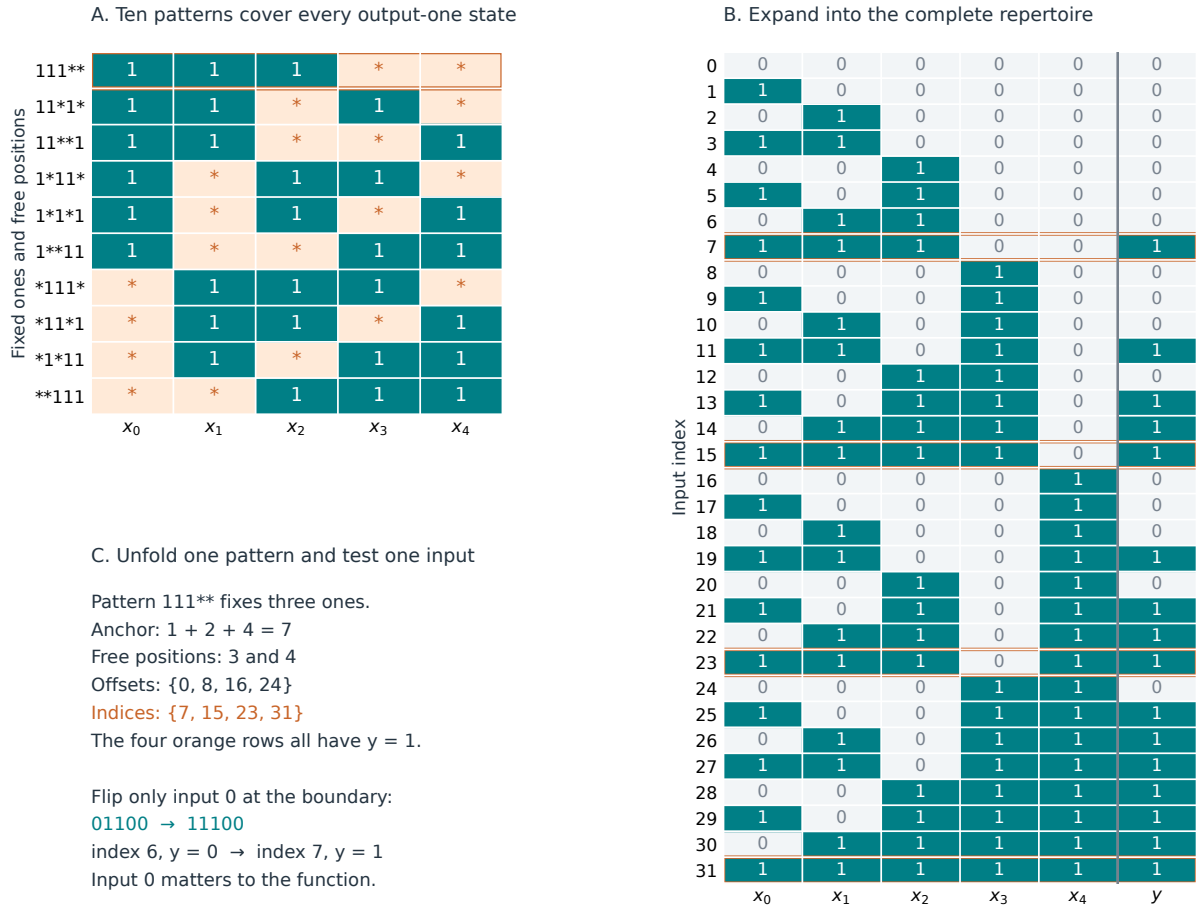


Figure 3: How do patterns recover the same output repertoire? Panel A shows the ten schemata for the network in Figure 2. Teal entries are fixed ones; orange stars may be zero or one. Panel B shows all 32 input states and the actual circuit output $y = g_6$, ordered by input index. Orange outlines identify the four fillings of $111**$. Panel C unfolds that pattern and shows a single-bit change that changes the output. The figure is generated from evaluated circuit states; the expanded schema union is checked against every output.

Free positions belong to a particular schema. In $111**$, inputs 3 and 4 can vary without changing the final answer. Both inputs still matter elsewhere in the repertoire. For example, fixing two of the other inputs to one makes flipping either one decisive. Likewise, changing 01100 to 11100 flips only input 0 and changes the output from zero to one. These paired states are what index deconvolution uses to recover functional dependence. The patterns and the perturbation pairs describe two complementary aspects of the same exact map.

For odd majority on n inputs, every input matters: fix the other $n - 1$ bits with exactly $(n - 1)/2$ ones, then flip the remaining bit. Complementation pairs each output-one state with an output-zero state, giving 2^{n-1} active indices. This explains why index deconvolution does not remove inputs from true majority, even though its output set admits schemata with free positions.

Recovery is not limited to circuits small enough to tabulate. Run on the decision diagram, deconvolution recovers the emitted circuit at 151 inputs: every one of the 151 inputs comes back essential, the gate comes back as majority, and the recovered object is identical to a threshold function built independently of the majority construction. That identity settles agreement on all 2^{151} input states at once, and not one of them is enumerated. The diagram holds 41,196,123 nodes for a circuit of 8,116,950 gates.

Table 1 lists the ladder. It grows as roughly n^4 , which is what lets the method climb: the running count of active inputs is all a majority needs to remember, so the description stays small as n grows. Not every function is so obliging, and Section 8.3 reports one that is not.

Table 1: Recovering majority from its own behaviour. Each row is the emitted circuit at that size, deconvolved and compared with an independently built threshold function. Every input is recovered as essential and every comparison is an identity, so the agreement holds on all 2^n states without enumerating any.

n	Gates	Decision nodes	States covered
11	205	1,473	2,048
31	14,190	77,573	2^{31}
63	245,086	1,273,757	2^{63}
101	1,623,175	8,304,498	2^{101}
151	8,116,950	41,196,123	2^{151}

What is recovered is the computed function, not a unique arrangement of internal gates. Several different circuits can share one repertoire.

Nor are an input count and an output frequency sufficient certificates. A majority of three disjoint three-input majorities uses all nine inputs and returns one on half its states, just like true nine-input majority. Yet groups 110, 110, 000 give local votes 1, 1, 0 and an incorrect final one: only four original bits are one. Comparing the actual index sets exposes this failure. Supplementary Section S4 gives the reconstruction proof, controls and limits of identifiability.

6 A Fourier transform within three levels (Q3)

A Fourier transform rewrites a vector as its frequency components. For N complex entries, use the unnormalised forward convention

$$(Fx)_r = \sum_{j=0}^{N-1} x_j e^{-2\pi i r j / N}. \quad (4)$$

The question permits vector additions, rotations and multiplications, but at most three multiplications along a path from an input to an output. A usual radix-two factorisation has $\log_2 N$ stages. I combine adjacent stages into three linear transforms, each requiring one multiplication level. The final transform also restores the natural frequency order.

To implement a linear transform on a packed vector, group its matrix entries by cyclic diagonal. Each diagonal contributes a rotation of the input, multiplied entrywise by a known vector of coefficients. Adding those contributions gives the output. Rotations can be shared: split each offset into a small step and a multiple of a larger step, reuse the small rotations and rotate each

grouped sum once. Supplementary Section S5 derives the identity and counts every operation, including the masks and rotations.

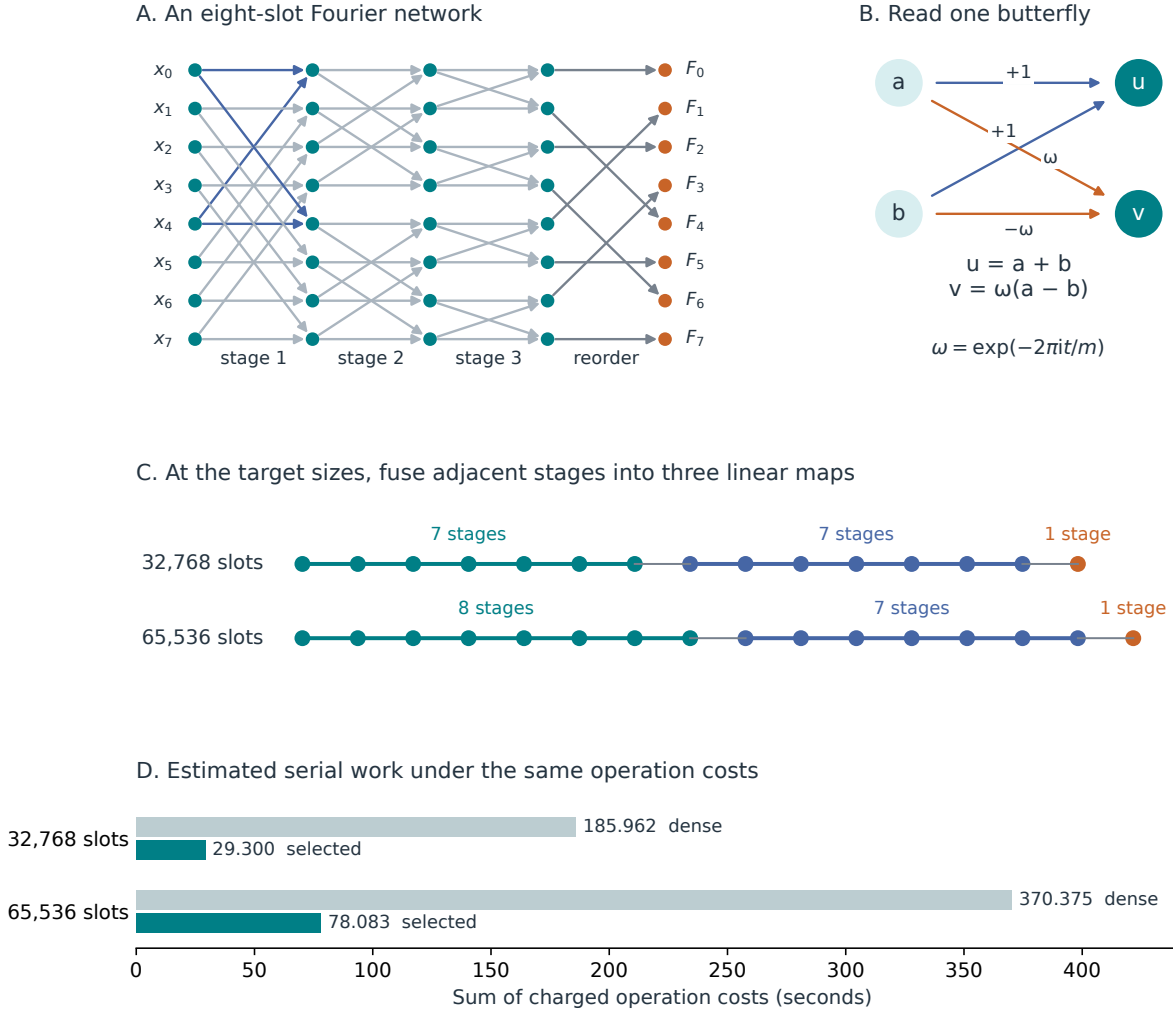


Figure 4: How does the Fourier network fit within three multiplication levels? Panel A illustrates the three radix-two stages for eight slots, followed by the permutation into natural frequency order. Panel B gives the butterfly rule: m is its block length and t its position within the half-block. Panel C shows the actual stage groupings at the two target sizes; each coloured group is evaluated as one fused packed linear map and consumes one multiplication level. The small network illustrates scalar dependencies, rather than the charged vector-operation count. Panel D compares estimated serial work under the same cost model; these are not ciphertext timings.

The source states 32,768 complex slots but uses 65,536 entries in its rotation example. I evaluate both sizes. Charging an addition at $4 \mu s$, a known-vector multiplication at $5600 \mu s$ and a rotation at $6100 \mu s$ gives the comparisons in Figure 4. Every multiplication by a known vector consumes one level under this convention.

The selected circuits use 4,757 multiplications, 4,754 additions and 433 rotations at 32,768 slots; the corresponding counts at 65,536 are 13,313, 13,310 and 570. Their estimated serial costs are 29.299516 and 78.083040 seconds. The dense costs are 185.962068 and 370.374740 seconds, respectively: 6.347 and 4.743 times as much estimated work.

All 106 and 121 contiguous partitions into at most three groups were examined, together with every power-of-two rotation grouping size. The choices minimise cost within this family. They do not establish a global optimum over all possible circuits. Executing every operation in plaintext on 13 test vectors per target size gives maximum normalised errors below 2.2×10^{-15} against an

independent Fourier transform. Ciphertext noise, key operations and encrypted timing require a separate implementation study.

7 When the modular equations force zero (Q4)

Let $q = 2^{127} - 1$, the stated prime, and suppose the sums of the first, second and third powers of a, b, c, d are all zero modulo q . There is no nontrivial solution. From the first equation, $d = -a - b - c$. Substitution into the third gives

$$a^3 + b^3 + c^3 + d^3 = -3(a + b)(a + c)(b + c) = 0. \quad (5)$$

Since a field has no zero divisors and $q \neq 3$, one pair sum must vanish. Relabel to obtain $b = -a$ and $d = -c$. The square equation now says $a^2 + c^2 = 0$. If $c \neq 0$, division by c^2 would make -1 a square. But $q \equiv 3 \pmod{4}$, so -1 is not a square in this field. Hence $c = 0$ and then all four entries are zero. Supplementary Section S6 supplies the short number-theoretic proof and small-prime examples showing why the assumptions matter.

8 Expressing integer conditions by quadratic equations (Q5–Q8)

These questions use a different, 254-bit prime P , specified in Supplementary Section S1. Arithmetic takes place modulo P . A constraint has the form $A(w)B(w) = C(w)$, where each expression is linear in the variables w . A *witness* is an assignment of all private and auxiliary values satisfying the constraints.

8.1 Direct equations and the CausalBool route

To require a 64-bit integer x (Q5), introduce 64 bits and impose

$$b_i(b_i - 1) = 0, \quad x = \sum_{i=0}^{63} 2^i b_i. \quad (6)$$

The first equation restricts each b_i to zero or one. Their weighted sum is below $2^{64} < P$, so it represents the intended integer without modular wraparound. To require $r \neq 1$ (Q6), add $(r - 1)s = 1$. Such an s exists exactly when $r - 1$ is nonzero: it is its multiplicative inverse.

For Q7, write the proposed factors as u, v , bound both to 64 bits and impose $uv = n$ for the public 64-bit integer n . Require each factor to be at least two. Since $uv < 2^{128} < P$, the modular product equation is also an integer equality. A valid factorisation supplies a witness, and a satisfying witness gives a valid factorisation. These direct constraints verify supplied factors; they do not provide an algorithm for finding them.

8.2 Building the same conditions out of recovered gates

The direct equations above are short because they borrow the field's own arithmetic. The second route asks a different question: if we are only allowed Boolean gates, which gates are they, and who chooses them? Here nothing is chosen by hand. Each cell is stated as an integer relation, its complete behaviour is handed to index deconvolution, and the gate that comes back is what gets compiled.

Handed the relation $a + b + c_{\text{in}}$ and nothing else, the method returns the sum bit as XOR on all three inputs and the carry bit as MAJORITY on all three. The classical full adder is recovered

rather than recalled. Handed the one-bit product it returns AND. Handed a running comparison it returns the comparator recurrence, and once the bound bit is fixed it specialises on its own: against a bound bit of zero it reports that the outgoing “less” depends on the incoming “less” alone, because no new strict inequality can arise there. That specialisation is found, not applied afterwards.

A recovered gate names a function of any arity, while the constraint layer carries two- and three-input gates, so the expansion between them is an obligation rather than a convenience. It is discharged by identity for every gate the method can return at the arities in use, and for every function of one to three variables.

Q5 is the range predicate $x < 2^{64}$ built from the recovered comparator, over a decomposition one bit wider than the bound so that out-of-range values are representable and only the comparator excludes them. Q6 uses the same comparator at 254 bits together with a recovered inequality cell. Q7 is a shift-and-add multiplier assembled from the recovered full adder and partial product, at the full 64 bits the question asks for: 107,271 rows over 53,572 signals. It accepts genuine factorisations including semiprimes whose product needs all 64 bits, and rejects trivial factors, wrong products, and a pair of 34-bit factors whose product overflows 64 bits and would otherwise pass as its truncated residue. Exhaustive enumeration is complete at the widths where it is possible: every one of the 299,008 triples at widths four, five and six agrees with an independent integer predicate.

This route is larger than the direct one everywhere, and no efficiency claim is made for it. What it buys is that the gates doing the arithmetic were derived from behaviour rather than asserted.

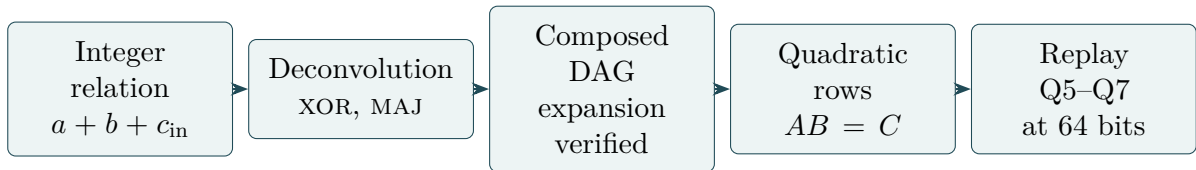


Figure 5: From a stated relation to verified constraints. The cell enters as an integer relation, not as a circuit. Deconvolution names its gates, the expansion into the constraint layer’s gate set is verified by identity, and the resulting rows are replayed against an independent predicate. No gate in this chain is written down by hand.

8.3 A large factorisation and the role of carries

A 4096-bit integer exceeds the field modulus, so one field element cannot hold it. For Q8, split each integer into 64 pieces, or *limbs*, in base $B = 2^{64}$. Each limb is a 64-bit integer:

$$u = \sum_{i=0}^{63} u_i B^i, \quad v = \sum_{i=0}^{63} v_i B^i, \quad n = \sum_{i=0}^{63} n_i B^i. \quad (7)$$

Multiply the limbs and collect terms by position, exactly as in written multiplication. For each column k ,

$$\sum_{i+j=k} u_i v_j + c_k = n_k + B c_{k+1}. \quad (8)$$

The incoming carry is c_k ; the outgoing carry is c_{k+1} . Set the initial and final carries to zero, and set output limbs above position 63 to zero. Figure 6 illustrates how the column equations fit together, using decimal digits so every partial product and carry can be followed by hand. The construction above uses the same pattern in base 2^{64} .

The bounds are part of the argument. Intermediate carries are restricted to 70 bits. Both sides of Eq. (8) are then below $2^{135} < P$, so the field cannot hide a wraparound in any column. Conversely,

Follow the partial products and carries: $23 \times 47 = 1081$ (base 10)

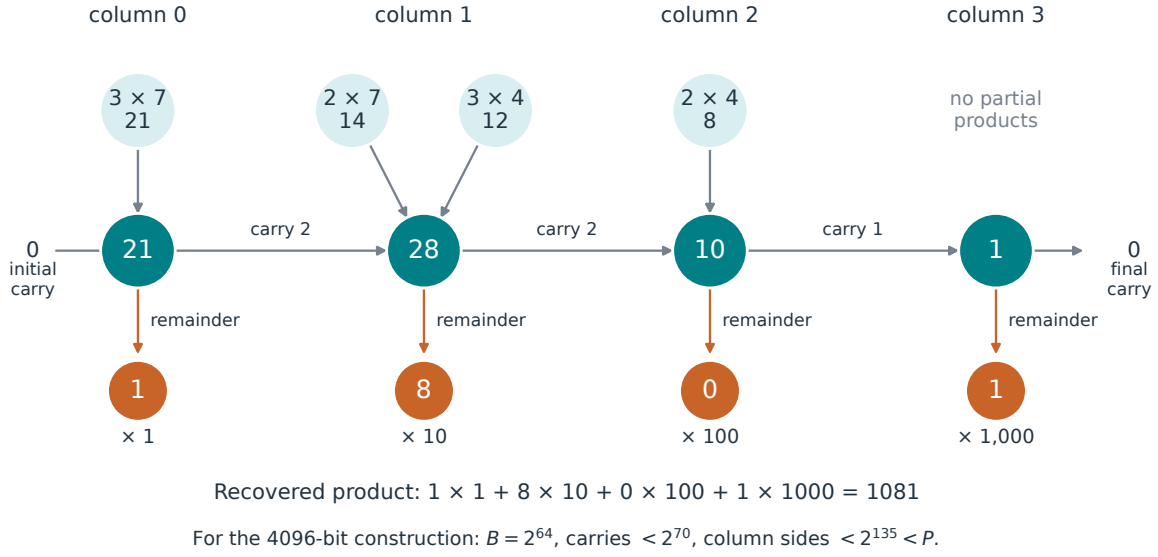


Figure 6: How do partial products become an exact integer product? The decimal example $23 \times 47 = 1081$ makes the carries visible. Each pale node is a partial product. Teal nodes add the products in one column and its incoming carry. Division by 10 sends the quotient to the next column and keeps the orange remainder as an output digit. Columns run from least to most significant, so the digits read 1, 8, 0, 1 in that order. The large construction uses the same equations in base 2^{64} ; its limb and carry bounds keep each column equality below the field modulus.

ordinary multiplication produces carries within these bounds. Together with nontrivial factors and the zero upper outputs, this proves the equivalence between a satisfying assignment and the intended integer factorisation.

Checks evaluate these equations directly, including altered bits, partial products, carries and public values. A useful negative control removes the carry bounds: the false integer statement $2((P + 7)/2) = 7$ can then satisfy the multiplication columns modulo P . Restoring the bounds rejects it. Supplementary Sections S7–S10 give completeness and soundness proofs and the compiled verification record.

8.4 Where the recovered route stops, and why

Q8 is answered above, and the recovered route of the previous subsection is not what answers it. It is worth being precise about why, because the reason is not effort.

The obvious attempt is to build a 4096-bit multiplier from the same recovered cells that give Q7. Its cost follows an exact law, fitted over widths four to sixty-four and confirmed at 128: a width- w multiplier has $7w^2 + 1$ gates and $26w^2 + 12w + 7$ rows. At $w = 4096$ that is 436,256,775 rows against 25,725 for the limb construction, a factor of about seventeen thousand. Large, but on its own only an argument about expense.

The real obstruction appears when we ask for the compressed description rather than the gate count. Deconvolution's reach comes from the decision diagram staying small, and whether it does is a property of the function, not of the implementation. Table 2 puts the two cases side by side on the same engine. Majority's description grows polynomially, as $n^{3.96}$ across six sizes, which is what carries it to 151 inputs. Multiplication's grows by a factor of 2.977 for every additional bit, measured across eleven widths, and is already beyond a budget of eight million nodes at thirteen bits. Extrapolating that factor to 4096 bits gives a description of some 10^{1942} nodes.

The observable universe holds about 10^{80} atoms.

Table 2: Why one function is recoverable at scale and the other is not. Both columns are the same engine on the same machine. A majority need only remember how many inputs are active, so its description stays polynomial. A multiplier must retain every partial sum, and no summary suffices; the growth is geometric and the limit is intrinsic.

	Majority	Multiplication
Description size	polynomial, $n^{3.96}$	geometric, $\times 2.977$ per bit
Measured over	6 sizes, $n = 31$ to 151	11 widths, $w = 2$ to 12
Largest built	41,196,123 nodes	4,384,542 nodes
At that size	$n = 151$, all 2^{151} states	$w = 12$ bits
Extrapolated	—	$\approx 10^{1942}$ nodes at 4096 bits

This is not a gap awaiting a better encoding. The exponential growth of such descriptions for integer multiplication is a known lower bound [7], and the measurements above are that bound appearing in our own engine. The limb construction avoids it entirely by never going through Boolean logic: each limb product is one field multiplication, and the blow-up has no opportunity to occur. Reporting the boundary is more useful than omitting the question, since it says exactly which problems this method is the right instrument for.

9 What this response establishes

The solutions give two-query recovery under rational arithmetic, a majority construction for every odd input count, a depth-three Fourier circuit at both stated dimensions, a proof that the modular power-sum solution is zero, and quadratic systems equivalent to the requested integer conditions. Their assumptions determine what each answer means.

The distinctive addition is the account of what survives each representation, and of who supplies each answer. Index deconvolution recovers functional dependence from behaviour and, run on a canonical decision diagram, does so without writing the repertoire down: majority comes back exactly at 151 inputs, and the arithmetic cells that build Q5, Q6 and Q7 come back from stated integer relations with no gate named by hand. The Fourier construction preserves the transform while making the price of a three-level restriction measurable under one declared model. The arithmetic construction preserves integer meaning inside a modular system by bounding every place where wraparound could occur.

The same account marks where the method does not reach. Q8 is answered by the limb construction and not by recovery, and the reason is measured rather than conceded: descriptions of integer multiplication grow geometrically in the width, so at 4096 bits no such description can be written down. A method worth using is one whose boundary is known, and this response reports both sides of it.

The finite evidence supports these specific constructions. It includes small majority circuits, full target-size plaintext Fourier evaluations and checks of compiled arithmetic constraints. A complete 2025-input majority circuit, encrypted Fourier timings and generated zero-knowledge proofs remain outside that evidence. Supplementary Information separates the general proofs from these finite results and provides the details needed to reproduce and assess them.

References

- [1] 0xPARC. *Beyond Primitive Computing: Puzzles and Connections*, Field Extension. <https://fielddextension.org/3b1b/>.
- [2] A. Hernández-Espinosa, H. Zenil, N. A. Kiani and J. Tegnér, *Estimations of Integrated Information Based on Algorithmic Complexity and Dynamic Querying*, arXiv:1904.10393 [q-bio.NC] (2019). <https://arxiv.org/abs/1904.10393>.
- [3] A. Hernández-Espinosa, H. Zenil, N. A. Kiani and J. Tegnér, “Estimations of Integrated Information Based on Algorithmic Complexity and Dynamic Querying,” in *Handbook of Unconventional Computing*, Vol. 1: Theory, ed. A. Adamatzky (World Scientific, 2021), Chap. 5, pp. 171–220. https://doi.org/10.1142/9789811235726_0005.
- [4] G. Cohen, I. B. Damgård, Y. Ishai, J. Kölker, P. B. Miltersen, R. Raz, and R. D. Rothblum. *Efficient Multiparty Protocols via Log-Depth Threshold Formulae*, 2013. https://www.wisdom.weizmann.ac.il/~ranraz/publications/Pmajority_mpc-1.pdf.
- [5] J. H. Cheon, K. Han, and M. Hhan. *Faster Homomorphic Discrete Fourier Transforms and Improved FHE Bootstrapping*. Cryptology ePrint 2018/1073. <https://eprint.iacr.org/2018/1073>.
- [6] A. Hernández-Espinosa, *CausalBool*, source code accompanying this response, 2026.
- [7] R. E. Bryant. *On the Complexity of VLSI Implementations and Graph Representations of Boolean Functions with Application to Integer Multiplication*. IEEE Transactions on Computers 40(2):205–213, 1991.
- [8] iden3. *Circom 2.2.3* and *snarkjs 0.7.6*. <https://docs.circom.io/>; <https://www.npmjs.com/package/snarkjs/v/0.7.6>.